Specifica, implementazione, verifica

L02 · Fondamenti e confronto

Mauro Leoncini

Obiettivi

- Distinguere **che cosa** deve fare un programma da **come** lo fa
- Scrivere proprietà (generali), non soltanto esempi (particolari)
- Usare la specifica come strumento per **giudicare** codice che non abbiamo scritto
- Capire che l'enunciazione (e la verifica) di una proprietà che il software deve soddisfare ha in generale un **costo** che è diverso nei diversi paradigmi

Lezione frontale + calcolatore + lavagna. Il codice generato dal modello si proietta, la specifica si scrive alla lavagna insieme agli studenti.

Filo: a L01 abbiamo detto che un programma ha un significato e un comportamento. Oggi aggiungiamo la terza cosa, che viene prima di entrambi: che cosa abbiamo davvero chiesto al modello?

Tre domande diverse

```
Che cosa deve fare?        →  specifica
Come lo fa?                →  implementazione
Lo fa davvero?             →  verifica
```

La terza domanda **non si può nemmeno porre** se manca la prima.

Un programma non è corretto o sbagliato in sé: è corretto **rispetto a una specifica**. Senza specifica, "questo codice è giusto" è una frase senza significato.

Il primo problema di oggi

Il prompt all'AI (copilot da VS Code):

> Mi serve un programmino python per calcolare il secondo massimo in una lista di interi.

È una frase in italiano corrente, facilmente comprensibile e tuttavia non priva di ambiguità.

Prima di scrivere una riga di codice, chiediamoci: **che cosa deve valere nel risultato?**

Risposte degli studenti

Vediamo le vostre proposte

1)

2)

...

- Che cosa succede se la lista è **vuota**? o se ha **un solo** elemento?
- ... o se il massimo compare **due volte**?

La frase non decide

Su `[5; 5; 3]`:

- **5** — il secondo elemento in ordine decrescente
- **3** — il secondo valore distinto

Due letture. Entrambe difendibili. La "specifica" in italiano non consente di optare in modo certo per nessuna della due!

Questa non è una lacuna del codice: è una **lacuna della specifica**, e nessuno può colmarla al nostro posto, neppure l'AI — perché dipende da che cosa ci serve.

Il codice generato

```python
def secondo_massimo(numeri):
    valori_distinti = sorted(set(numeri), reverse=True)

    if len(valori_distinti) < 2:
        raise ValueError("La lista deve contenere almeno ".\
                         "due valori distinti")

    return valori_distinti[1]
```

Leggibile, con un controllo sugli ingressi, con un nome di funzione sensato.

Qual è la domanda giusta?

Chiedersi ora «il codice è corretto?» **non è** ragionevole.

> **Quale delle nostre proprietà viola, e con quale ingresso lo dimostriamo?**

La risposta deve essere un **controesempio**: un ingresso e il valore sbagliato che ne esce. Un'opinione non è una risposta.

Due specifiche possibili

A — Secondo valore distinto

Precondizione: la lista contiene almeno due valori distinti.

Postcondizione: il risultato è il secondo valore distinto in ordine decrescente.

```
[3, 1, 2]  →  2
[5, 5, 3]  →  3
```

B — Secondo elemento ordinato

Precondizione: la lista contiene almeno due elementi.

Postcondizione: il risultato è il secondo elemento della lista ordinata in senso decrescente.

```
[3, 1, 2]  →  2
[5, 5, 3]  →  5
```

La correttezza è condizionale

Un contratto separa due responsabilità:

```
chiamante  → deve soddisfare la precondizione
funzione   → deve garantire la postcondizione
```

Un ingresso che viola la precondizione non è, da solo, un controesempio alla correttezza della funzione.

> Prima di cercare un controesempio dobbiamo sapere quali ingressi appartengono al dominio della specifica.

Che cosa abbiamo imparato sul modello

Il modello ha scritto codice ragionevole per **una** delle due letture, e **potrebbe non aver detto quale**.

Anche nel caso fosse stato "silenzioso", non avrebbe mentito né sbagliato: ha completato in un modo ragionevole una richiesta incompleta, che è esattamente il suo mestiere.

> La decisione mancante era nostra prima dell'AI e resta nostra con l'AI.

Un secondo esperimento

Il prompt all'AI (ancora copilot da VS Code):

> Mi implementi una classe C++ che gestisce un 2–3 tree, implementando al momento solo inserzione e ricerca? Mi implementi anche un main interattivo per inserimento e richiesta in modo da poter fare delle prove?

Copilot produce la classe e un programma di prova.

Inserzione e ricerca funzionano correttamente sui casi provati.

Che cosa non dice il prompt?

Il prompt chiede di implementare `insert`, ma non ne specifica completamente il significato.

> Che cosa deve accadere quando inseriamo una chiave già presente?

Alcune possibilità:

- ignorare il nuovo inserimento;
- conservare più occorrenze della chiave;
- segnalare un errore;
- restituire un esito che distingua inserimento e duplicato.

La prima scelta implicita

Il programma impedisce l'inserimento di una chiave già presente.

```
insert(30)
insert(30)

contenuto dell'albero: 30
```

Copilot ha interpretato il 2–3 tree come rappresentazione di un **insieme**, non di un multiinsieme.

È una scelta ragionevole.

Non era però scritta nel prompt.

Due contratti diversi

Se l'albero t rappresenta un insieme, l'inserimento è idempotente:

$$insert(x, insert(x, t)) \equiv insert(x, t)$$

Se rappresenta un multiinsieme, la molteplicità deve invece aumentare:

$$m_{insert(x,t)}(x) = m_t(x) + 1$$

Lo stesso programma è corretto rispetto al primo contratto e scorretto rispetto al secondo.

Il secondo prompt

> Ora vorrei che alle chiavi fosse associata un'informazione, diciamo una semplice stringa. Puoi aggiornare?

Anche questa richiesta viene soddisfatta: ogni chiave è ora associata a una stringa e la ricerca restituisce l'informazione corrispondente.

Ma il nuovo prompt non stabilisce che cosa debba accadere inserendo la stessa chiave con una nuova informazione.

Stessa chiave, nuova informazione

Eseguiamo concettualmente:

```
insert(30, "prima")
insert(30, "seconda")
search(30)
```

Il programma restituisce:

```
"seconda"
```

La nuova informazione ha sostituito la precedente.

Copilot è passato implicitamente dalla semantica di un insieme a quella di una mappa con aggiornamento.

Quale comportamento volevamo?

Interpretazione	Secondo inserimento della stessa chiave
Mappa con sostituzione	sostituisce la vecchia informazione
Mappa senza sostituzione	conserva la prima informazione
Inserimento controllato	segnala il duplicato
Multimappa	conserva entrambe le informazioni
Aggregazione	combina vecchia e nuova informazione

La scelta corretta dipende dall'applicazione, non dall'albero 2–3.

Il codice è sbagliato?

Non abbiamo ancora elementi per dirlo.

Il programma ha incorporato una decisione progettuale ragionevole che il prompt non aveva preso.

> Il test con una chiave duplicata non è ancora un controesempio alla correttezza del codice: è un controesempio alla completezza della richiesta.

Solo dopo aver scelto il contratto possiamo giudicare il comportamento.

La decisione cambia l'interfaccia

Se ogni chiave identifica una sola informazione, la ricerca può restituire:

```
std::optional<std::string> search(int key);
```

Se una chiave può identificare più informazioni, occorre qualcosa come:

```
std::list<string> searchAll(int key)
```

Se i duplicati devono essere rifiutati, anche `insert` deve comunicare l'esito:

```
bool insert(int key, std::string value)
```

Proprietà diverse per contratti diversi

Mappa con sostituzione

$$search(insert(k, v, t), k) = v$$

Multimappa

$$v \in searchAll(insert(k, v, t), k)$$

e le informazioni già associate a k devono essere conservate.

Inserimento senza sostituzione

Se k è già presente, l'albero e l'informazione associata non cambiano.

Che cosa ha fatto davvero l'AI?

Quando la richiesta non determina completamente il comportamento, il modello produce comunque un programma concreto.

Per farlo, completa la richiesta usando convenzioni e scelte plausibili.

> Ragionevole non significa necessariamente adatto alla nostra applicazione.

La generazione del codice non elimina le decisioni mancanti: le rende **implicite nel programma**.

Adesso vogliamo di più

Provare qualche ingresso non equavale a dimostrare. Nel caso del secondo max con `[5, 5]` abbiamo trovato **un** difetto; ma non sappiamo se ce ne sono altri.

Che cosa servirebbe per essere sicuri?

Bisogna iniziare anche a chiedersi con precisione, **cosa** bisogna fare e **quanto costa**.

Cominciamo da indagare

Un programma di ordinamento

```
void ordina(std::vector<int>& a);      // ordina sul posto
```

Qual è o quali sono le post-condizioni?

«Prima» non esiste più

Al termine del metodo il contenuto iniziale di `a` è stato **sovrascritto**. Non c'è nessuna espressione C++ che lo denoti.

Per enunciare la proprietà servono i linguaggi di specifica, che aggiungono un costrutto apposta:

```
/*@ ensures ordinato(a);
  @ ensures permutazione(a, \old(a)); */
```

`\old(a)` non è una funzione: significa *«valuta* `a` *nello stato precedente alla chiamata»*. Il linguaggio delle asserzioni deve saper parlare di **due istanti diversi**.

ACSL per C, usato da Frama-C; JML per Java; Dafny come linguaggio a sé. Nominarli e basta: servono a mostrare che l'apparato esiste ed è reale, non a studiarli.

La stessa proprietà, senza tempo

```
ordina : int list -> int list
```

Dopo `let s = ordina l`, la lista `l` **non è stata distrutta**: `l` e `s` esistono entrambi, nello stesso istante, ed entrambi hanno un nome.

```
ordinata (ordina l)  &&  permutazione l (ordina l)
```

Nessun costrutto nuovo, nessun riferimento al tempo, nessuna copia: solo l'ingresso, l'uscita e l'applicazione di funzione che il linguaggio già ha.

E non è una questione di linguaggio

La stessa cosa, in C++, senza uscire dal linguaggio:

```
std::vector<int> ordinata(std::vector<int> a) {   // per valore
    ordina(a);
    return a;
}
```

```
ordinato(ordinata(v))  &&  stessi_elementi(v, ordinata(v))
```

Scrivibile, corretta, e nessuno la scrive: si paga una copia, e l'idioma del linguaggio spinge verso la versione che modifica.

> La differenza non è fra C++ e OCaml. È fra **modificare** e **restituire**, e si vede dentro un solo linguaggio.

Il secondo costo: l'invariante

Nella versione imperativa la correttezza si dimostra sul **ciclo**, non sulla funzione: bisogna trovare una proprietà che valga a ogni giro.

Ma una tale proprietà nomina uno **stato intermedio** che nella specifica non compariva affatto — un accumulatore, un indice, un prefisso già elaborato.

- la specifica parla del **problema**
- l'invariante parla di **quel particolare programma**

Due programmi corretti per la stessa specifica hanno invarianti diversi.

Detto onestamente

I programmi imperativi **si verificano formalmente**, eccome: JML e KeY per Java, Frama-C per C, Dafny, e con questi si verifica software industriale.

La tesi non è «non si può». È piuttosto:

> serve un **apparato in più** prima ancora di poter dire che cosa si vuole

E in questo corso quell'apparato non lo studieremo: sceglieremo linguaggi in cui la proprietà si scrive con quello che c'è già.

La scala della verifica

```
esempio          "su [3;1;2] restituisce 2"
   ↓
proprietà        "il risultato è un elemento della lista"
   ↓
controesempio    la macchina cerca l'ingresso che la rompe     (L09)
automatico
   ↓
dimostrazione    vale per OGNI ingresso, e lo so perché        (L08)
```

Oggi siamo al secondo gradino, e ci siamo saliti a mano.

Che cosa portate a casa oggi

- «Questo codice è corretto» è una frase incompleta: corretto **rispetto a che cosa**
- Un esempio non è una proprietà
- Il modello generativo completa richieste incomplete: la decisione mancante resta vostra
- La forma del linguaggio decide **quanto costa enunciare** una proprietà, prima ancora di dimostrarla

Consegna, per L03

> Data una lista di stringhe, restituisci la più lunga.

Una pagina, senza codice: **tre proprietà**, **che cosa la frase non dice** con le vostre decisioni, e **un ingresso** su cui sospettate che il modello si comporti diversamente da voi.

Poi, e solo poi, generate il codice e provate quell'ingresso.

Testo completo in `codice/consegna.md`.